

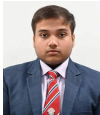


Graphic Era
deemed to be **University**
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

MiniGit: A Lightweight Version Control System Using C++ and Data Structures

Team Name: Team #	Blaze DSCPP-III-2026-T296
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	Gangwar, Abhinesh – 2510370192 2510370192@geu.ac.in 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	Jain, Sparsh – 2510370159 2510370159@geu.ac.in 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	Bajaj, Krish – 2510370469 2510370469@geu.ac.in 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	Joshi, Snehil – 2510370146 2510370146@geu.ac.in 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

MiniGit is a lightweight, local, command-line version-control system written in C++. Its goal is to explain the core idea of version control in a simple form: create a repository, track files, save versions with commit messages, view history, and restore earlier work. The project applies C++, OOP, file handling, and data structures in one practical tool.

In Phase 2, the team built the working repository and commit modules. A user can now create a repository with init, stage files with add and remove, block unwanted files through a .minigitignore file, save versions with commit, move between saved states with undo and redo, and read the history with log.

Each feature is built around one data structure: a Dual Trie for ignore rules, a set for the staging area, a deque for undo and redo, and a doubly linked list for commit navigation. The four modules were developed in separate branches and joined through one command dispatcher. The remaining work is restore (checkout and rollback), a stronger storage format, more testing, and documentation.

Updated Project Approach and Architecture

We designed MiniGit around a clean, 5-layer modular architecture:

1. **The CLI Interface:** Reads and checks the command.
2. **Command Handler:** Sends it to the right module and turns the result into a message and exit code.
3. **Repository and staging manager:** Handles the index, commits, and undo history.
4. **Ignore engine:** Decides which files are blocked.
5. **Storage layer:** Reads and writes the .minigit folder.

Every module returns a plain result object (AddResult, IgnoreResult, CommitResult) and never prints. Only the CLI prints, so each module is easy to test.

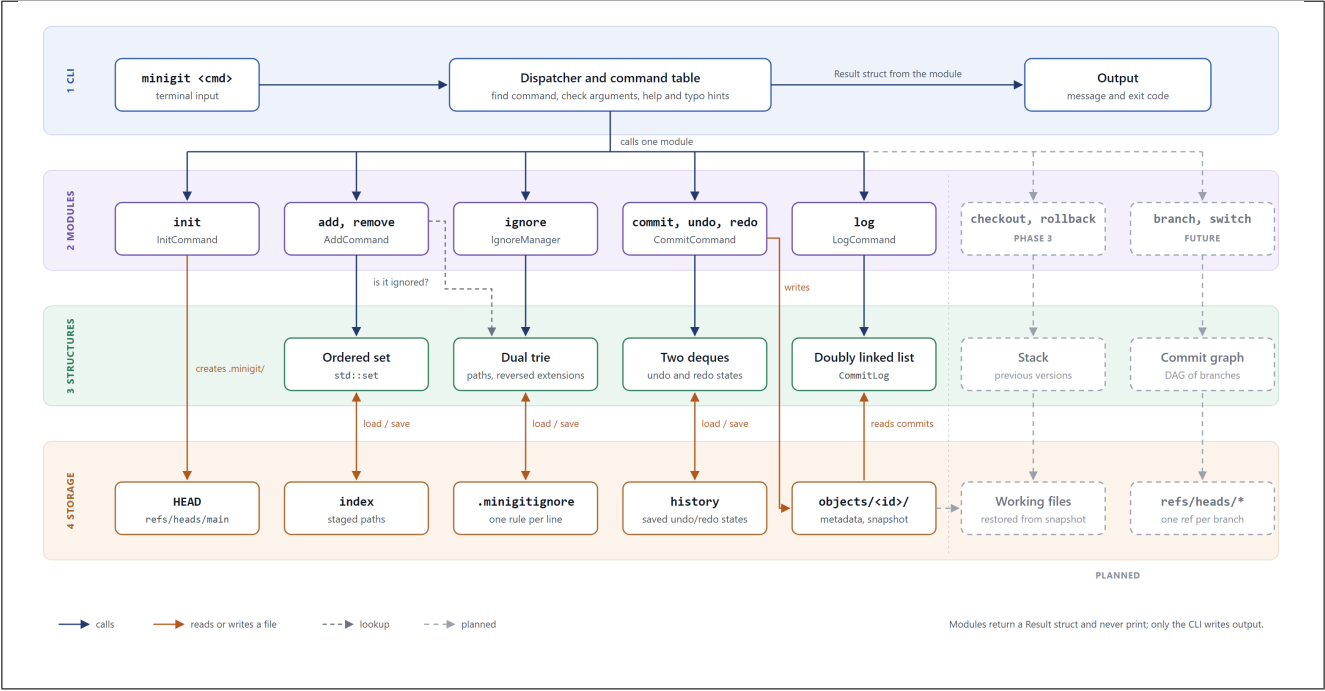
Updated data structures:

- **Dual Trie:** a forward Trie for file and directory rules, and a reversed Trie for extensions like *.exe. An ignore check takes $O(L)$ time for a path of length L .
- **Set:** `std::set` holds the staging index and prevents duplicates.
- **Deque:** stores undo and redo states; the oldest state is dropped when the history limit (20) is reached.
- **Doubly linked list:** each commit links to its older and newer commit, so log works in both directions.

Changes from Phase 1: the stack became a deque, the singly linked list became a doubly linked list, and a Trie-based ignore engine was added.

Storage and libraries: .minigit holds HEAD, index, the history file, and objects/<commit-id>/ with metadata and snapshots. Commit IDs are 40-character SHA-1 hashes from a self-contained implementation. Only the C++ standard library is used. `std::filesystem` comes with a bundled header-only fallback, so the code builds on older compilers (C++14/C++17) with no external dependency.

Updated Architecture Diagram



Tasks Completed

Task Completed	Team Member
Repository setup with init and the .minigit layout (HEAD, index, objects); CLI parser and dispatcher; ignore command interface with input checks; result-to-message mapping and exit codes; first-run install so minigit runs from any folder, and Documentations	Abhinesh Gangwar
Dual Trie (insert, search, remove, isIgnored); .minigitignore loading with line numbers; add <file>, add ., add -v, remove; staging index with duplicate prevention; ignore -r and ignore -v; tests for add and ignore	Sparsh Jain
commit "<message>" with SHA-1 commit ID, metadata, parent link, and snapshots; deque-based undo and redo with history limit; commit tests	Krish Bajaj
Doubly linked list (insert, find, next, prev); history loading from commit metadata; log and log --reverse showing ID, message, and timestamp; log tests	Snehil Joshi

Challenges/Roadblocks

- **Path differences:** Windows backslashes and . prefixes broke matching. A shared normalizePath() now converts every path to one form.
- **Extensions in a prefix tree:** a Trie matches prefixes, but extensions are suffixes. Extensions are stored reversed in a second Trie.
- **Wrong directory matches:** build/ must not block buildings/. Directory rules keep their trailing slash.
- **Line numbers after removal:** deleting a rule shifts the lines below it. The file is rewritten and both Tries are rebuilt.
- **Joining four separate modules:** branches could drift apart. The team agreed early on result objects, the index format (one path per line), and CLI-only printing.
- **Undo and redo consistency:** HEAD and the staged files must change together. Each deque entry stores the commit ID and a snapshot of the staged files.
- **Compiler support:** older compilers lack std::filesystem, so a bundled header-only fallback is used.
- **Open issue:** tests for init and the CLI are still missing and are planned next.

Tasks Pending

Task Pending	Team Member
checkout: restore a selected commit from stored snapshots	Snehil Joshi, Krish Bajaj
rollback: reverse the latest restoration using a stack of recent states	Krish Bajaj
Sample repository, demo walkthrough, README update	Sparsh Jain
Phase 3 roadmap: branching, DAG history (with log --graph), branch switching, ancestor-based merge	All members
Final report and live demonstration	All members

Project Outcome/Deliverables

The final outcome will be a working terminal-based MiniGit application in C++ for local text files. The core operations are `init`, `add`, `remove`, `ignore`, `commit`, `undo`, `redo`, `log`, `checkout`, and `rollback`.

The submission will include organized C++ source code, a sample repository, test cases, a README and user guide with command examples, a project report, and a live demonstration.

The application builds into one standalone executable with no external dependency, and a first-run step lets the user type `minigit` from any terminal. The branching, checkout and rollback features are planned extensions that depend on time and the stability of the core.

Progress Overview

Four of the six core operations in the proposal (`init`, `add`, `commit`, `log`) are complete and tested. This is about two-thirds of the core scope. All four member modules are finished and joined through one command dispatcher.

Ahead of schedule: the Trie-based ignore engine, undo and redo with a deque, the `remove` command, and first-run installation. None of these was required at this stage.

Behind schedule: checkout and rollback, which belong to the restore stage.

In progress: tree objects, and packfile compression.

Codebase Information

Repository Link: <https://github.com/strange-exe/MiniGit>

Documentation Site: <https://abhinesh.codes/MiniGit/>

Active Branch: main holds the integrated code. Each member works in an individual branch and merges into main through a pull request after tests pass.

Layout: module/include, module/src, module/tests, Reports/, README.md, DOCUMENTATION.md

Important Commits:

- Repository setup init, CLI parser, and dispatcher (Abhinesh)
- Trie, Ignore Manager, Staging Area, add / remove / ignore, tests (Sparsh)
- commit with SHA-1 IDs, deque-based undo and redo (Krish)
- Doubly linked list (CommitLog) and log, tests (Snehil)
- All documents merged into main, plus README and documentation (Abhinesh)

Testing and Validation Status

Test Type	Status	Notes
Unit test: Trie (test_trie)	Pass	Insert, search, remove; node pruning; extension matching; build/ does not match buildings/.
Unit test: add and ignore (test_add)	Pass	Staging, duplicate prevention, add rule line numbers, CRLF files, saved index.
Unit test: commit (test_commit)	Pass	18 checks: 40-character ID, metadata, parent link, undo, redo, history limit.
Unit test: log (test_commitlog, test_log)	Pass	List insert, find, next, prev; newest-first and -reverse order; log after undo; empty repository.
Manual CLI run	Pass	Full flow on a sample folder: init, ignore, add, commit, undo, redo; errors for bad input.

Deliverables Progress

- **Documentation Site** (<https://abhinesh.codes/MiniGit/>): Completed
- **Core CLI Pipeline** (init, add, commit, log, checkout): Completed
- **Dual Trie Ignore Engine** (.minigitignore): Completed
- **Deque Undo/Redo Integration**: Completed
- **Branching DAG & 3-Way Merge**: Pending (Phase 3)